Типы данных, определяемые пользователем

enum – struct – union

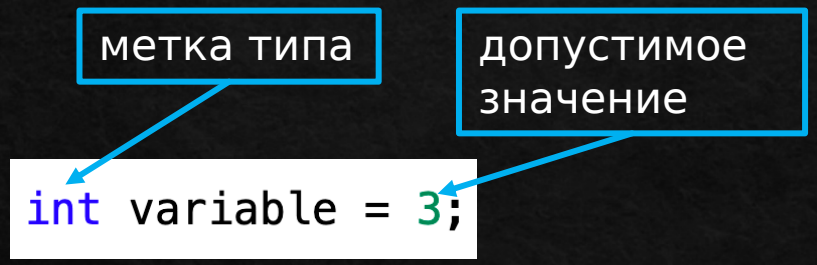
Тип

Что такое тип?

Тип – синтаксическая метка, приписываемая некоторому элементу программы

Система типов – синтаксический метод доказательства отсутствия в программе определённых видов поведения

Бенджамин Пирс «Типы в языках программирования»



абсурдное выражение

```
auto ridiculous = variable + "Hello";
```

сигнатура функции является ограничивающей использование синтаксической меткой

```
int f(int*,unsigned)
```

абсурдное выражение

```
auto ridiculous = f(1.f,-1.f);
```

Тип данных

Чаще всего тип данных определяется как совокупность множества допустимых значений и операций (функций), которые могут быть к нему применены.

Группа по модулю 3

e 1 2

e + 1 = 1 | e + 2 = 2 | e + e = e
1 + 1 = 2 | 1 + 2 = e
2 + 2 = 1

Дни недели

MON
Seq(MON) -> TUE
Seq(TUE) -> WED
Seq(WED) -> THU
…
Seq(SUN) -> MON

Предопределённые типы данных

Множества значений и операции над этими типами, а равно и их преобразования, определены стандартом языка и работать с ними можно без дополнительных средств.

```
signed / unsigned char
signed / unsigned short (int)
signed / unsigned (int)
signed / unsigned long (int)
signed / unsigned long long (int)

float
double

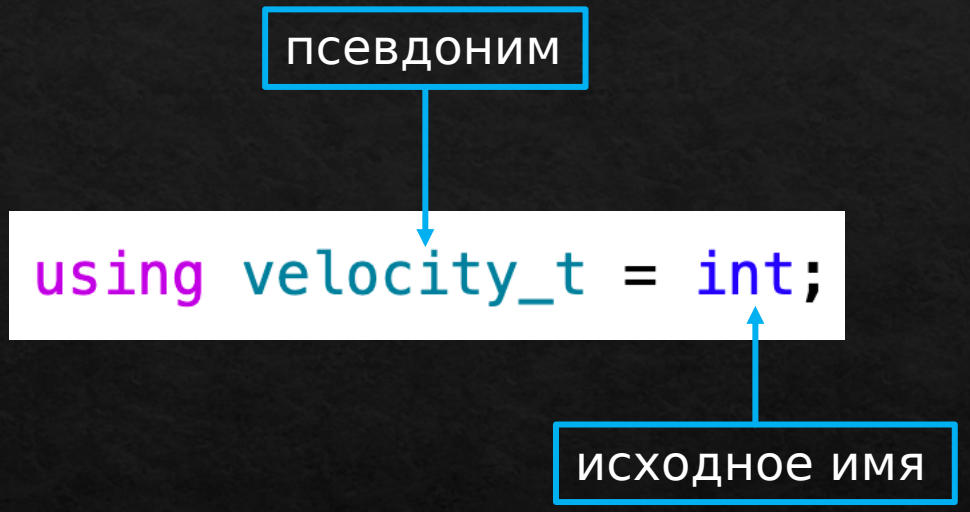
void *
```

Переименования

typedef & using

using

Ключевое слово using позволяет ввести более удобное имя (псевдоним) для уже существующего типа.




```
int main() {
    velocity_t v1 = 2, v2 = -3;
    auto v_sum = v1 + v2;
    return 0;
}
```

using

С псевдонимами следует быть осторожным! Они не вводят новых ограничений и потому разные псевдонимы одного типа могут использоваться взаимозаменяемо. Это может приводить к абсурдным выражениям!

```
 2  using velocity_t = int;
 3  using distance_t = int;
 4  using interval_t = int;
 5
 6  int main() {
 7      velocity_t v = 3;
 8      interval_t t = 10;
 9      distance_t s = v * t;
10
11      auto ridiculous = s - t / v;
12  }
```

не возникнет ошибок или предупреждений, так как вычисления проходят в типе int

auto разрешит тип переменной как int

typedef

Ключевое слово typedef сохранилось как часть языка C и также выполняет функцию объявления псевдонимов, но имеет ограничение: не позволяет создавать переименования для «неспециализированных шаблонов».

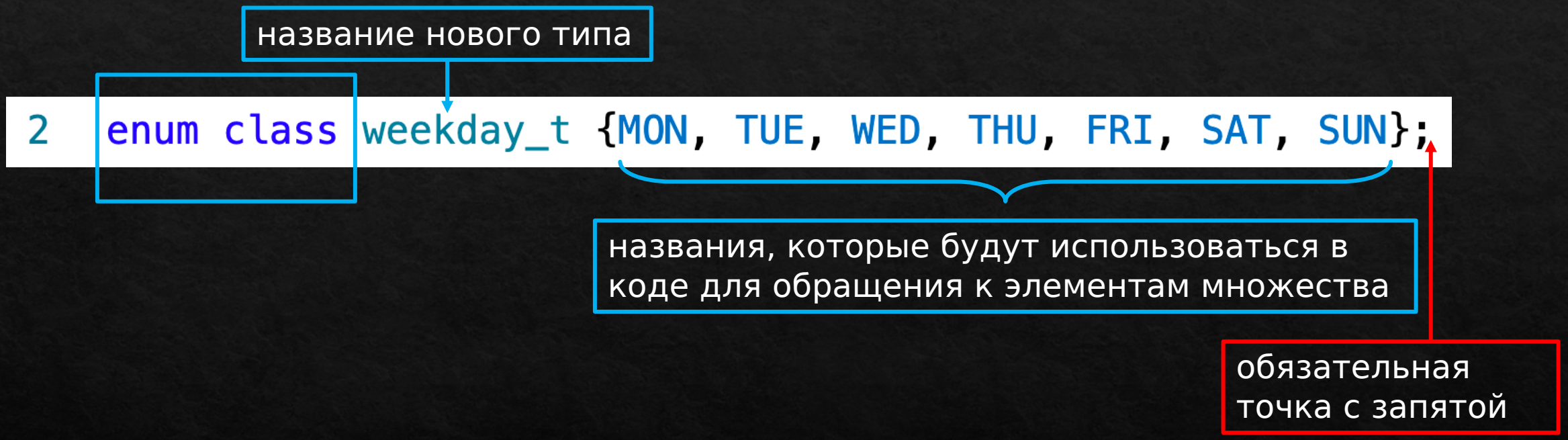


Перечисления

enum class

enum class

Перечисление – это тип данных, построенный над множеством из небольшого количества значений. «Под капотом» этот тип данных использует значения типа int.



weekday_t

Какой сегодня день недели?

```
2   typedef int weekday_t;
3
4   int main() {
5       weekday_t today = 1;
6       auto next_day = today + 7;
7       return 0;
8   }
```

Какой день получится?

сегодня понедельник

```
2   enum class weekday_t {MON, TUE, WED, THU, FRI, SAT, SUN};
3
4   int main() {
5       weekday_t today = weekday_t::MON;
6       auto next_day = today + 7;
7       return 0;
8   }
```

выражение неверно

```
tmp.cpp:6:27: error: invalid operands to binary expression ('weekday_t' and 'int')
    auto next_day = today + 7;
                    ~~~~~ ^ ~
```

weekday_t

```
 7  weekday_t next_day(weekday_t day) {
 8      if (weekday_t::SUN == day) return weekday_t::MON;
 9      return static_cast<weekday_t>(1 + static_cast<int>(day));
10  }
11
12  weekday_t operator+(weekday_t day, unsigned day_interval) {
13      auto diff = day_interval / 7;
14      weekday_t result = day;
15      switch(diff) {
16          case 6:
17              result = next_day(result);
18          case 5:
19              result = next_day(result);
20          case 4:
21              result = next_day(result);
22          case 3:
23              result = next_day(result);
24          case 2:
25              result = next_day(result);
26          case 1:
27              result = next_day(result);
28      }
29      return result;
30  }
```

```
32  int main() {
33      weekday_t today = weekday_t::MON;
34      auto next_day = today + 8u;
35      cout << static_cast<int>(next_day) << endl;
36      return 0;
37  }
```

N Значения enum class преобразуются в int явным образом. По умолчанию нумерация элементов идёт с нуля. Так же возможно и обратное явное преобразование.

Следует иметь в виду, что преобразование из int в enum class всегда успешно, даже если значения с таким номером не существует.

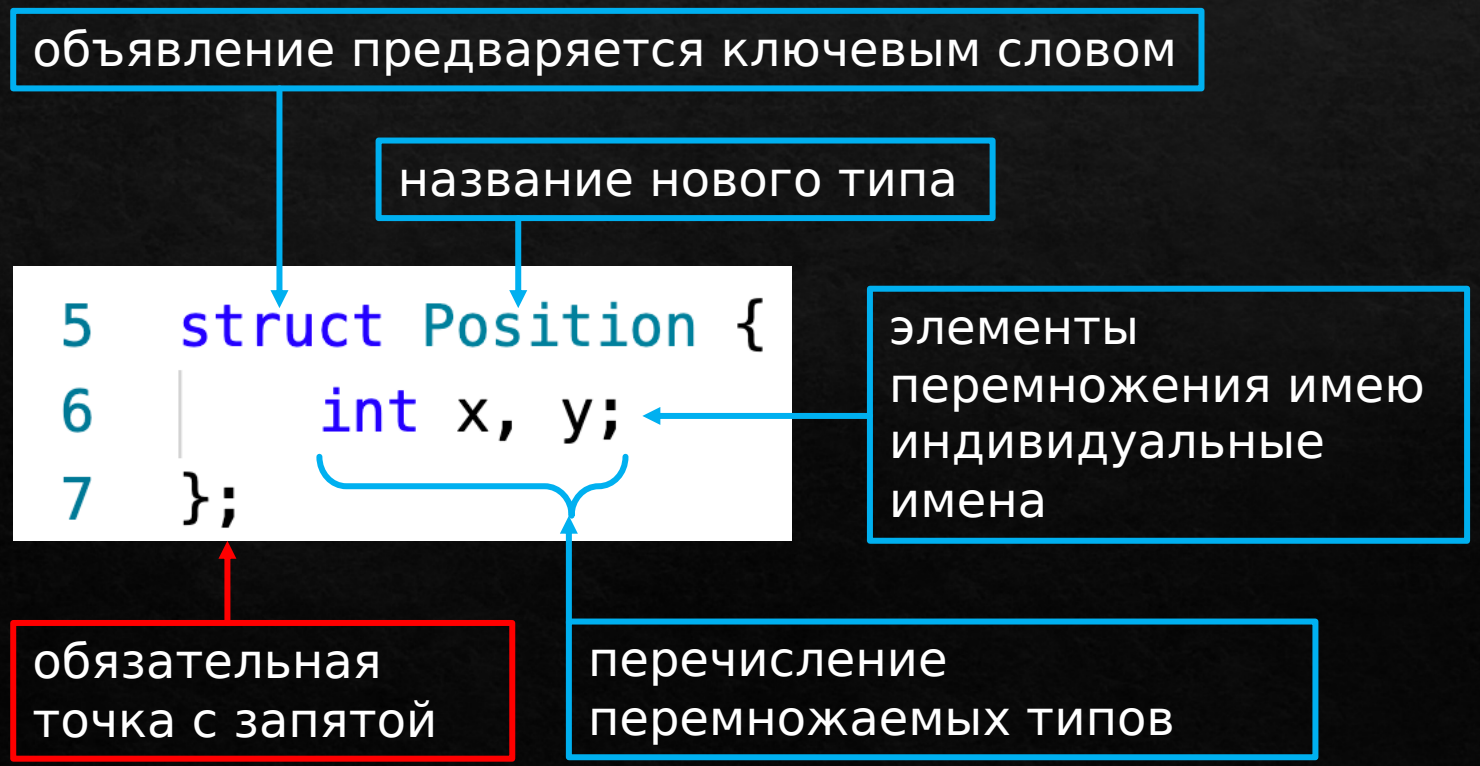
Структуры

struct

Перемножение типов

Если рассматривать типы, как множества, то можно рассмотреть декартово произведение множеств и определить таким образом новый тип данных. Например, точку на плоскости можно задать парой координат. Тогда множество всевозможных точек – это декартово произведение типа одной координаты, на тип второй координаты.

Для определения таких типов в языке C++ используется ключевое слово struct.



Использование структур

```
 5  struct Position {
 6      int x, y;
 7  };
 8
 9  int main() {
10      Position p;
11      p.x = 1;
12      p.y = -1;
13
14      p.x += 2;
15      p.y += 3;
16
17      cout << p.x << " " << p.y << endl;
18      // cout << p << endl; // error
19
20      return 0;
21  }
```

объявление переменной с типом структуры

обращение к отдельным элементом перемножения

над отдельными элементами структуры можно производить все действия, допустимые для тех типов, которым они принадлежат

для самого типа структуры необходимо определять операции дополнительно

Использование структур

```
 9  Position move(Position initial, int x_move, int y_move) {
10      Position result;
11      result.x = initial.x + x_move;
12      result.y = initial.y + y_move;
13      return result;
14  }
```

```
16  Position center(Position lha, Position rha) {
17      Position result;
18      result.x = (lha.x + rha.x) / 2;
19      result.y = (lha.y + rha.y) / 2;
20      return result;
21  }
```

Инициализация структур

```
23  void print(Position const &p) {
24      cout << '(' << p.x << ',' << p.y << ')' << endl;
25  }
26
27  int main() {
28      Position
29          p1 = {1, -1},
30          p2 = {.x = 2, .y = 3},
31          p3 = {.y = 1, .x = -1};
32
33      print(p1);  (1,-1)
34      print(p2);  (2,3)
35      print(p3);  (-1,1)
36
37      return 0;
38  }
```

Инициализация структур по умолчанию

```
5   struct Position {
6       int x = 1, y = -1;
7   };
```

```
13  int main() {
14      Position p1, p2 = {-1, 1};
15
16      print(p1); (1,-1)
17      print(p2); (-1,1)
18
19      return 0;
20  }
```

Когда важна инициализация?

```
 5  struct ConstPosition {
 6      int const x, y;
 7  };
 8
 9  struct Body {
10      unsigned const mass;
11      ConstPosition &pos;
12  };
13
14  int main() {
15      ConstPosition cp = {1, -1};
16      Body b = {.mass = 1u, .pos = cp};
17  }
```

элементы структуры неизменяемы

элементы структуры – ссылки

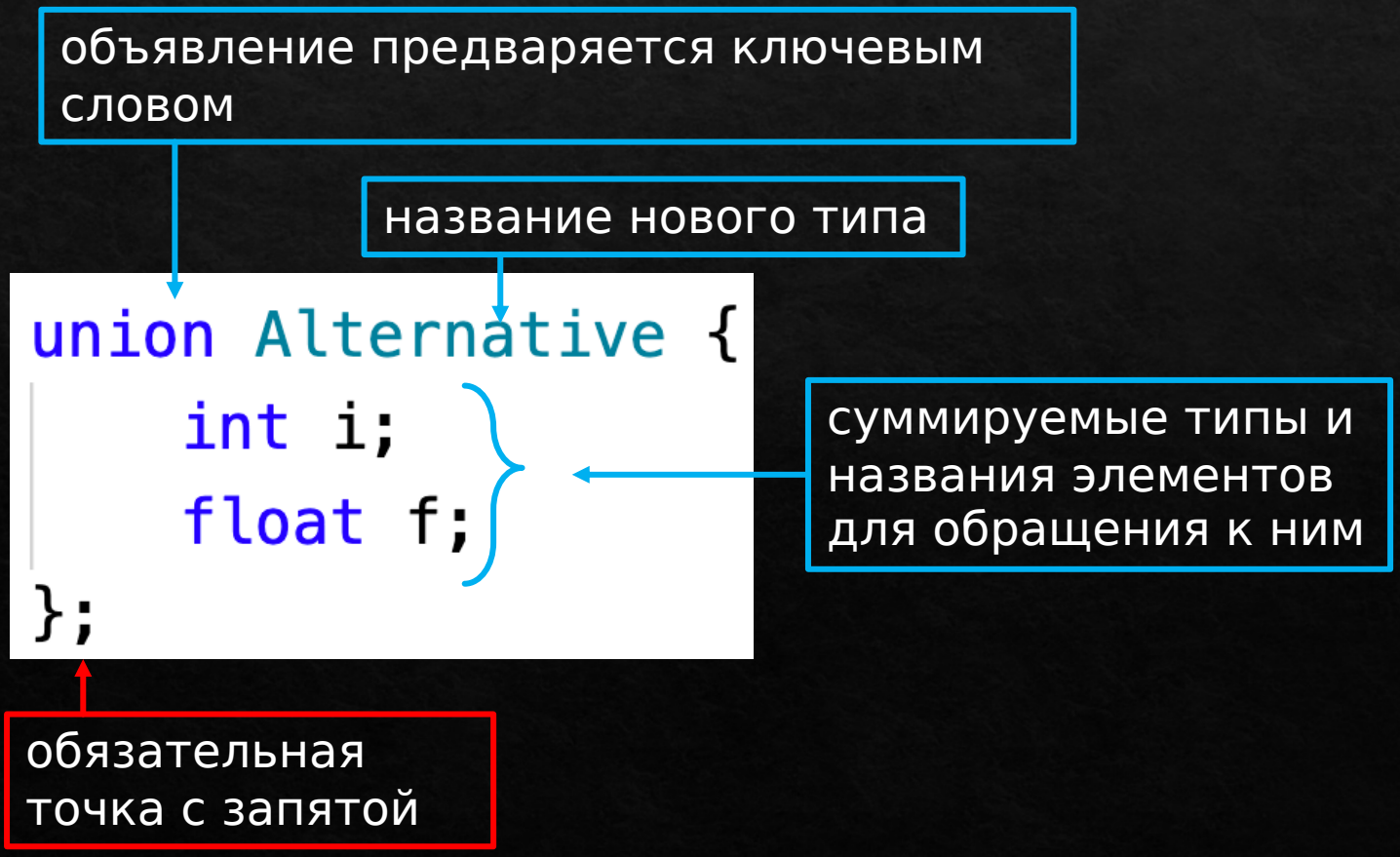
Объединения

union

Сумма типов

Типы как множества можно объединять. Такая операция называется «сумма типов». Элемент суммы типов может содержать любой элемент из любого суммированного типа.

Для определения таких типов в языке C++ используется ключевое слово union.



Использование объединений

```
21  union Cell {
22      int i;
23      float f;
24  };
25
26  int main() {
27      cout << setprecision(9) << scientific;
28      Cell c;
29      c.i = 10;
30      cout << c.i << " " << c.f << endl;
31
32      c.f = 1.f;
33      cout << c.i << " " << c.f << endl;
34
35      return 0;
36  }
```

```
10 1.401298464e-44
1065353216 1.000000000e+00
```

Следует отметить, что после того, как элементу присвоено некоторое значение, нет никакой возможности узнать какой именно элемент содержится в объединении.

Инициализация объединений

```
26  int main() {
27      cout << setprecision(9) << scientific;
28      {
29          Cell c = {.i = 10};
30          cout << c.i << " " << c.f << endl;
31      }
32      {
33          Cell c = {.f = 1.f};
34          cout << c.i << " " << c.f << endl;
35      }
36
37      return 0;
38  }
```

```
10 1.401298464e-44
1065353216 1.000000000e+00
```

std::variant

```
 4  #include <variant>
 5
 6  int main() {
 7      std::variant<int, float> cell;
 8      cell = 10;
 9      std::cout << std::get<int>(cell) << std::endl;
10
11      cell = 1.23f;
12      std::cout << std::get<float>(cell) << std::endl;
13
14      try {
15          std::cout << std::get<int>(cell) << std::endl;
16      } catch (std::bad_variant_access const &ex) {
17          std::cerr << ex.what() << std::endl;
18      }
19      return 0;
20  }
```

```
10
1.23
bad_variant_access
```